

HOSPITAL MANAGEMENT SYSTEM

A Java + MySQL console application

PROJECT BY SHRIYA MISTRY





INTRODUCTION

The Hospital Management System is a Java-based console application that helps a hospital manage patients, doctors and appointments in one place. It replaces paper registers with a simple, menu-driven program backed by a MySQL database.

The system is built around three classes – Patient, Doctor and Management – and supports adding and viewing patients, viewing doctors and booking appointments.

Java | MySQL | JDBC

PROBLEM STATEMENT

Hospitals and small clinics often rely on paper registers or scattered spreadsheets to record patients and schedule doctors. This leads to lost or duplicate records, no quick way to look up doctors, and – most importantly – the same doctor being booked twice for one time slot.

This project builds a basic hospital management system that stores patient and doctor data in a database and books an appointment only when the doctor is free.



FUNCTIONAL REQUIREMENTS

- Patient Management
- Doctor Management
- Appointment Booking
- Menu-driven Interface

Core features

PATIENT MANAGEMENT

- Add a new patient with details
- View all registered patients
- Get a patient's ID for booking

DOCTOR MANAGEMENT

- View doctors stored in MySQL
- Get a doctor's ID for booking

APPOINTMENT BOOKING

- Book using patient ID, doctor ID and time slot
- Booking fails if the slot is already taken

MENU & EXIT

- Add / view patients, view doctors, book, exit
- Menu repeats until the user exits

NON-FUNCTIONAL REQUIREMENTS

01

Performance

Menu actions and database queries respond within about 2 seconds for typical hospital data.

02

Usability

Simple numbered menu with clear prompts, so a new user can learn it in minutes.

03

Reliability

Data is stored persistently in MySQL, invalid input is handled without crashing, and double-booking is prevented.

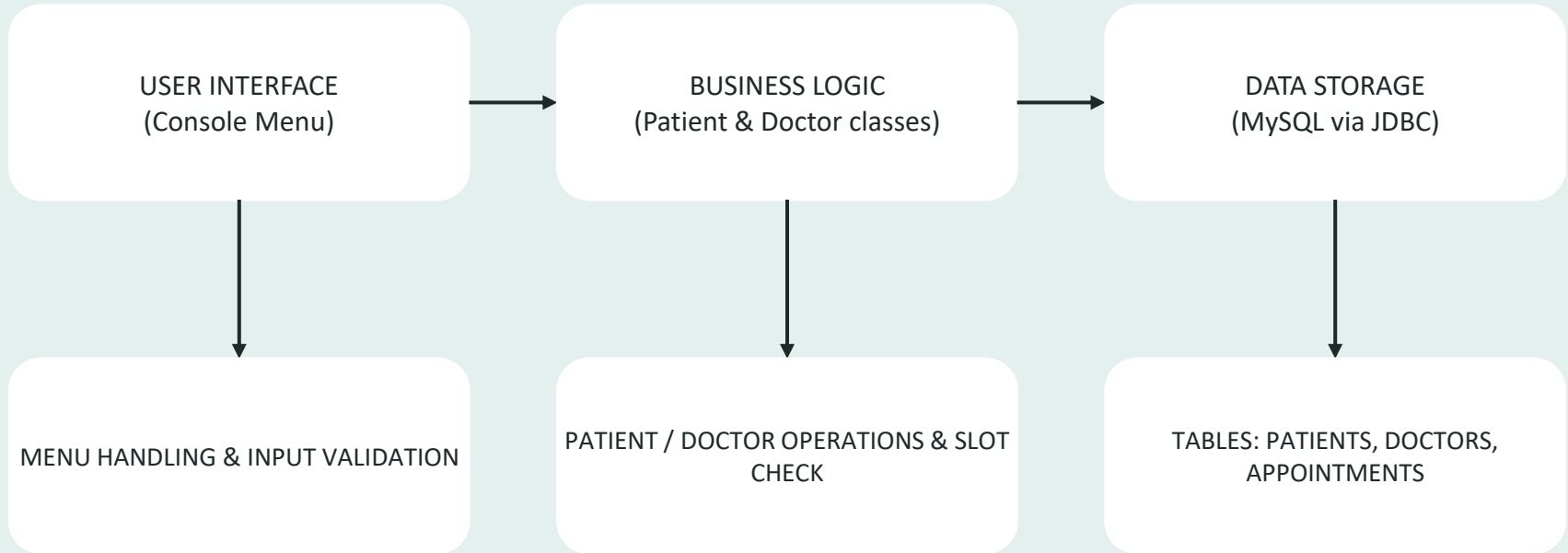
04

Maintainability

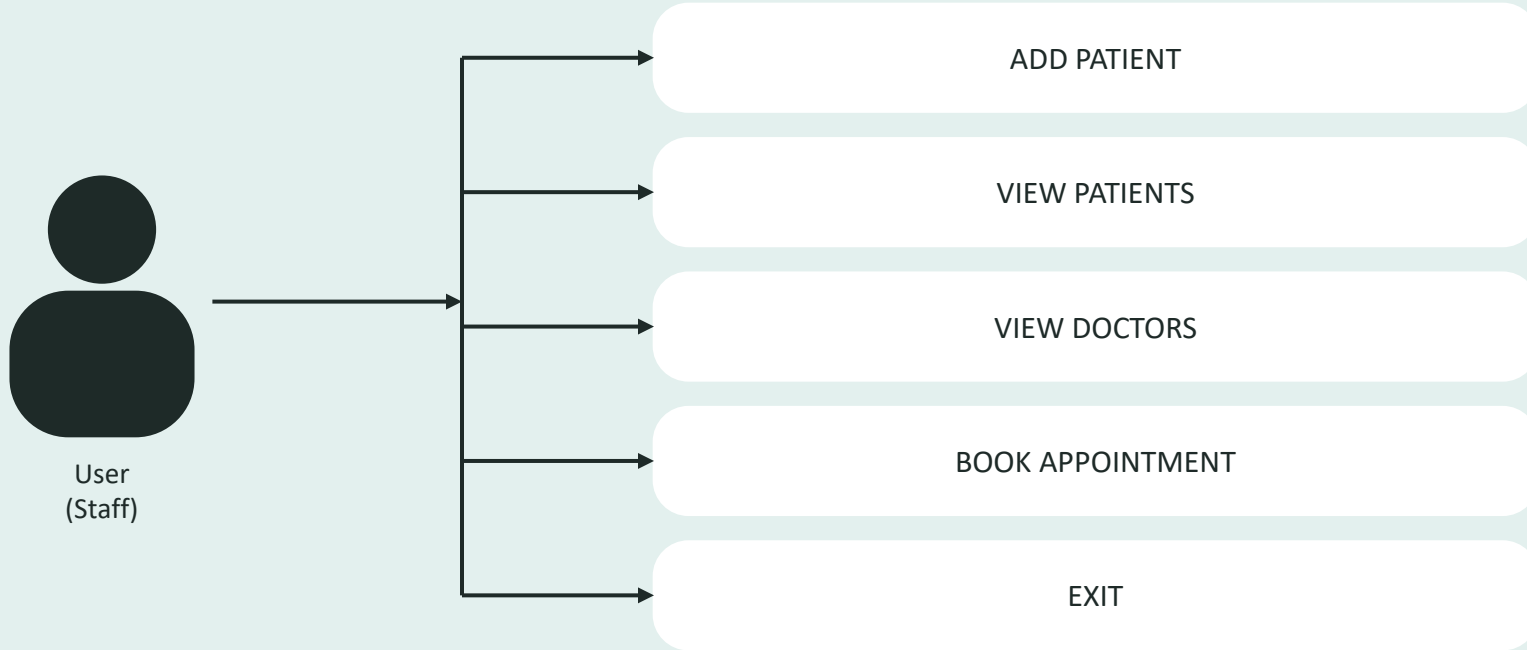
Three separate classes make the code easy to read, and new menu options can be added easily.



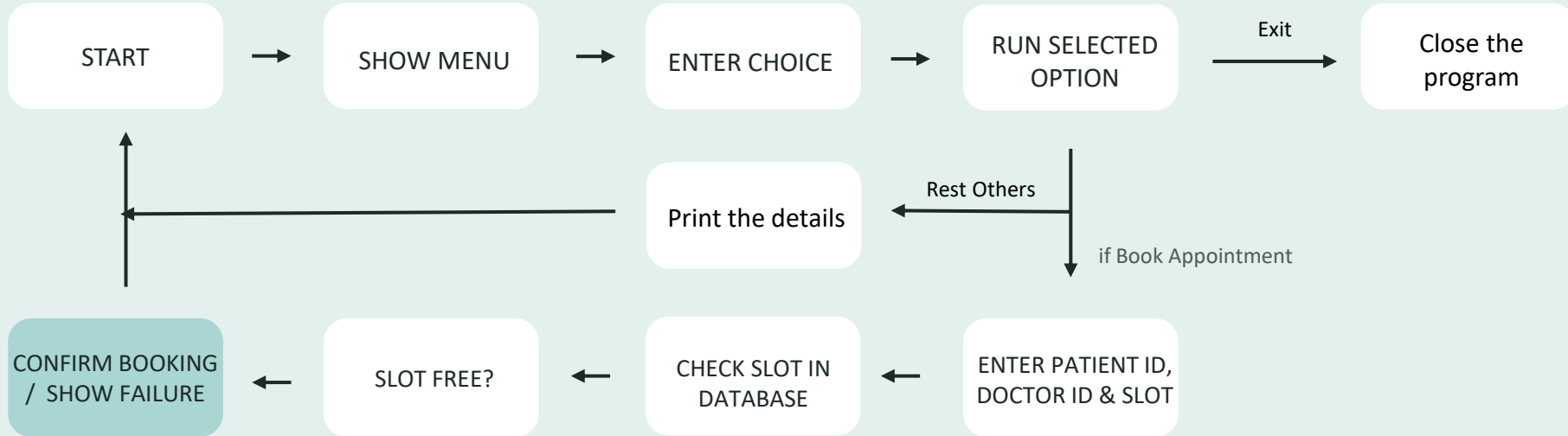
SYSTEM ARCHITECTURE



USE-CASE DIAGRAM

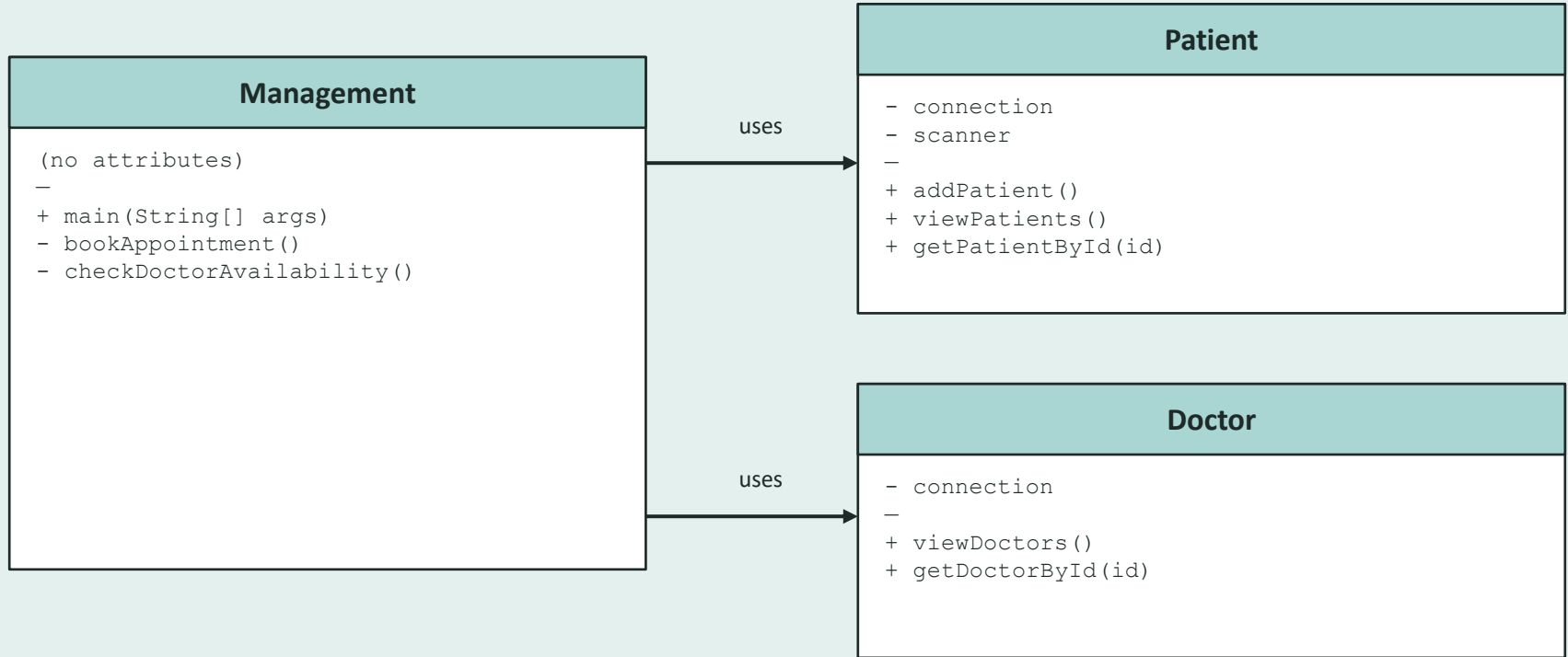


WORKFLOW DIAGRAM

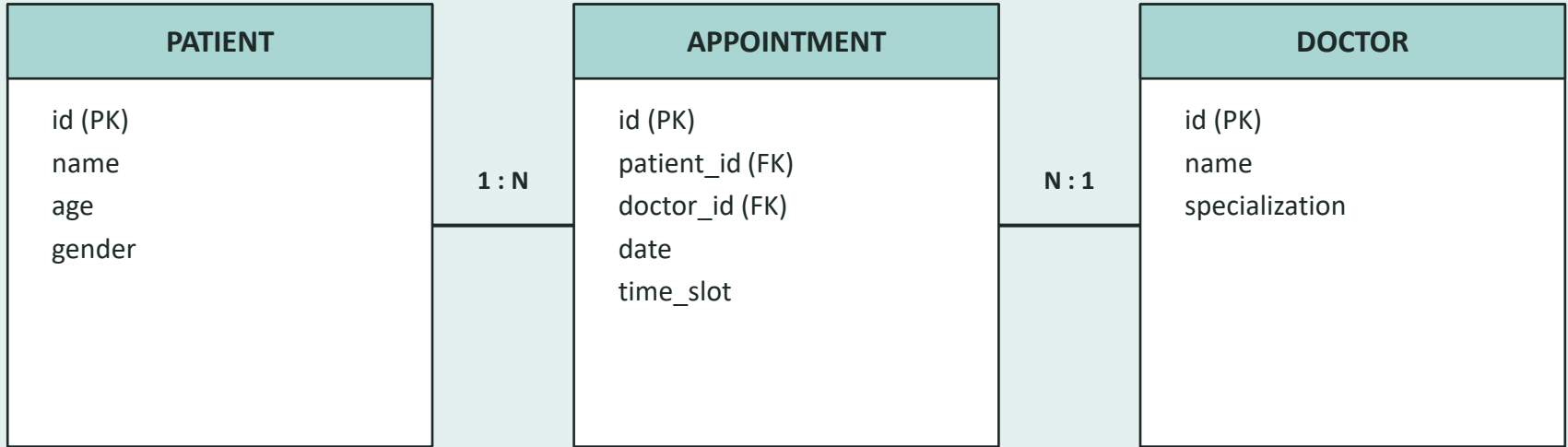


After every action the program returns to the menu until Exit is chosen.

CLASS DIAGRAM



ER DIAGRAM



Rule: one doctor cannot have two appointments with the same date and time slot.

DESIGN DECISIONS & RATIONALE

THREE-CLASS SPLIT

- Patient, Doctor and Management each have one job
- Keeps code clean and easy to extend

MYSQL DATABASE

- Data stays saved after the program closes
- Doctors can be added without touching code

JDBC + PREPARED STATEMENTS

- Safe, parameterised queries
- Avoids SQL injection errors

CHECK SLOT BEFORE BOOKING

- Doctor + time is verified first
- Guarantees no double-booking

IMPLEMENTATION DETAILS

1. Create MySQL database and add doctor records
2. Connect Java to MySQL using JDBC
3. Patient class: add, view, get patient ID
4. Doctor class: view doctors, get doctor ID
5. Management class: menu loop and appointment booking with slot check



**JAVA
+
MySQL**

SCREENSHOTS / RESULTS

Successful booking

```
====HOSPITAL MANAGEMENT====  
1. Add Patient  
2. View Patients  
3. View Doctors  
4. Book Appointment  
5. Exit  
Enter choice: 4  
Enter Patient ID: 3  
Enter Doctor ID: 2  
Enter date: 2026-09-25  
Enter time slot: 10:00  
  
Appointment Booked Successfully!
```

Doctor already booked

```
Enter choice: 4  
Enter Patient ID: 5  
Enter Doctor ID: 2  
Enter date: 2026-09-25  
Enter time slot: 10:00  
  
Doctor is not available at this slot!  
Appointment booking failed.
```

TESTING APPROACH

Manual black-box testing of every menu option with valid, invalid and boundary inputs.

Test case	Input	Expected result
Add patient	Valid details	Saved in database
View doctors	Option 3	Doctor list from MySQL
Book free slot	Valid IDs, free time	Appointment confirmed
Book taken slot	Same doctor and time	Booking fails
Invalid choice	Menu number 9	Asks user to retry

CHALLENGES FACED

- Technical Challenges
- Design Challenges

TECHNICAL CHALLENGES

- JDBC connection and driver setup
- Preventing double-booking of a doctor
- Handling wrong or empty user input
- Reading correct IDs from the database

DESIGN CHALLENGES

- Deciding what each class should own
- Designing the appointment table
- Keeping the menu simple to use
- Reusing one database connection

LEARNING & KEY TAKEAWAYS

1

TECHNICAL LEARNING

Connecting Java to MySQL with JDBC, writing SQL from code, and applying object-oriented design with separate classes.

2

PROJECT MANAGEMENT

Breaking the project into small steps, building one feature at a time, and testing each before moving on.

3

PERSONAL DEVELOPMENT

Problem-solving, attention to edge cases such as double-booking, and patience while debugging.

FUTURE ENHANCEMENTS

SHORT-TERM

- Swing / JavaFX GUI
- Cancel or reschedule appointments
- Update and delete patients
- Search patients by name

MEDIUM-TERM

- Login for admin, doctor, staff
- Billing and payments
- Reports and statistics
- Email / SMS reminders

LONG-TERM

- Web or mobile app (Spring Boot)
- Electronic health records
- Cloud deployment
- AI-based smart scheduling

REFERENCES

1. Oracle, The Java Tutorials – docs.oracle.com/javase/tutorial
2. Oracle, JDBC Basics – docs.oracle.com/javase/tutorial/jdbc
3. MySQL 8.0 Reference Manual – dev.mysql.com/doc
4. MySQL Connector/J Developer Guide – dev.mysql.com/doc/connector-j
5. GeeksforGeeks, Java JDBC and OOP tutorials – [geeksforgeeks.org](https://www.geeksforgeeks.org)

Thank you very much!

PRESENTED BY SHRIYA MISTRY